AquaSense: Monitor Inteligente de Umidade do Solo

Documentação Técnica (padrão README) · Inovatech 2026 · Grupo 7

I. Cabeçalho e Identificação

Campo	Valor
Nome do Sistema	AquaSense: Monitor Inteligente de Umidade do Solo
Nicho temático	Agricultura / irrigação automatizada (vertical **Umidade do Solo**)
Versão	**v1.1** (firmware ESP32, placa real). A v1.0 é a planta de execução em C puro, homologada em 03/11/2026.
Grupo	Grupo 7
Integrantes	Danilo Martinelli Bueno Veiga Manoel Florenço da Silva Júnior Rian Godinho Caixeta Oliveira Martins Yan Nascimento Sousa
Repositório público	https://github.com/Danilexxx/aquasense-inovatech
Demonstração on-line	https://danilexxx.github.io/aquasense-inovatech/

II. Escopo Arquitetural

Problema físico mapeado: Solo

A irrigação feita "no olho" causa dois erros que matam a planta:

- **Falta de água:** o solo chega ao **ponto de murcha**, e a raiz não consegue mais retirar água.
- **Excesso de água:** o solo passa da **capacidade de campo**, a água escorre, a raiz apodrece e há desperdício.

O AquaSense mede continuamente a umidade do solo com um sensor capacitivo e aciona uma bomba d'água sozinho, mantendo o solo dentro da faixa ideal.

Grandeza medida e calibração

O sensor entrega uma tensão que o ADC de 12 bits da ESP32 converte em um número de **0 a 4095**. **Quanto mais água, menor o número.** Esse valor bruto é convertido em **percentual de umidade (grau de saturação)** por interpolação linear em dois trechos, com calibração própria feita com 3 potes da mesma terra vermelha (argilosa), em 11/11/2026:

Ponto de calibração	ADC bruto	Umidade
Solo seco	4095	0%
Solo ideal	2411	50%
Solo encharcado	1277	100%

```
/* Trecho seco -> ideal (0 a 50%) e trecho ideal -> encharcado (50 a 100%) */
if (adc >= ADC_IDEAL) pct = (ADC_SECO - adc) * 50.0f / (ADC_SECO - ADC_IDEAL);
else                  pct = 50.0f + (ADC_IDEAL - adc) * 50.0f / (ADC_IDEAL - ADC_MOLHADO);
```

Métrica de acionamento lógico (histerese)

Condição	Ação
Umidade **< 30%** (`LIMITE_SECO`) e bomba desligada	**Liga** a bomba (relé no GPIO26)
Umidade **> 60%** (`LIMITE_ENCHARCADO`) e bomba ligada	**Desliga** a bomba
Entre 30% e 60%	**Mantém** o estado atual

A banda morta entre 30% e 60% é a **histerese**. Sem ela, com um único limite, o ruído do sensor faria a bomba ligar e desligar várias vezes por segundo (efeito *chattering*), desgastando o relé e o motor. Com dois limites, a bomba trabalha em ciclos longos e estáveis.

A classificação do solo para exibição segue os mesmos limites: **SECO** (< 30%), **IDEAL** (30% a 60%) e **ENCHARCADO** (> 60%).

Esteira de processamento

```
Sensor (GPIO35) --média de 16 amostras--> struct Leitura --enfileirar--> FILA (FIFO)
                                                                            |
                                                          a cada 3 leituras: desenfileirar
                                                                            v
                    PILHA (LIFO) <--empilhar evento-- Regra de negócio (histerese)
                                                                            |
                                                               Relé da bomba (GPIO26) + LED (GPIO2)
```

III. Lista de Materiais (BOM)

Qtd	Componente	Especificação	Função no sistema
1	**ESP32 DevKit V1**	ESP-WROOM-32, dual-core 240 MHz, ADC de 12 bits, Wi-Fi/Bluetooth, conversor USB-serial CP210x	Microcontrolador: lê o sensor, executa a esteira FIFO/LIFO e aciona o atuador
1	**Sensor capacitivo de umidade do solo v1.2**	Alimentação 3,3 a 5 V, saída analógica, sem eletrodo metálico exposto	Sensor: mede a umidade do solo
1	**Módulo relé 1 canal**	Bobina 5 V, contato NA (normalmente aberto)	Atuador: chaveia a bomba
1	**Mini bomba d'água submersa**	3 a 5 V DC	Atuador final: irrigação
1	**LED de status**	LED embutido da placa (GPIO2)	Indica bomba ligada
1	**Cabo USB**	Cabo de dados (não só de carga)	Alimentação, gravação e canal serial
—	**Jumpers e protoboard**	Fios macho-fêmea e macho-macho	Interligação
3	**Potes de calibração**	Mesma terra vermelha: seco, ideal e encharcado	Calibração do sensor
1	**Computador com Google Chrome**	Web Serial API	Painel de monitoramento (`medidor.html`)

Custo estimado das peças: **R$ 80 a R$ 120**.

IV. Engenharia de Memória (Inovação)

1. Mapeamento das variáveis dentro das `struct`

Tamanhos na ESP32 (arquitetura de 32 bits: `int`, `float`, `enum` e ponteiro ocupam 4 bytes cada).

`Leitura`: a unidade de dado que trafega pela esteira (16 bytes)

Campo	Tipo	Bytes	Significado
`id`	`int`	4	Número sequencial da leitura
`adc_bruto`	`int`	4	Valor cru do ADC (0 a 4095)
`umidade`	`float`	4	Umidade convertida (0 a 100%)
`status`	`StatusSolo` (`enum`)	4	`SOLO_SECO`, `SOLO_IDEAL` ou `SOLO_ENCHARCADO`

`No`: nó da lista encadeada da fila, alocado no **heap** com `malloc` (20 bytes)

Campo	Tipo	Bytes	Significado
`dado`	`Leitura`	16	A leitura armazenada
`proximo`	`struct No *`	4	Ponteiro para o próximo nó (`NULL` no último)

`Fila`: descritor da fila FIFO (12 bytes)

Campo	Tipo	Bytes	Significado
`inicio`	`No *`	4	Primeiro nó (de onde se remove)
`fim`	`No *`	4	Último nó (onde se insere). Mantê-lo permite inserir em O(1)
`tamanho`	`int`	4	Quantidade de nós (limite `CAP_FILA = 10`)

`Evento`: registro de acionamento da bomba (12 bytes)

Campo	Tipo	Bytes	Significado
`id_leitura`	`int`	4	Leitura que causou o evento
`umidade`	`float`	4	Umidade no momento do evento
`acao`	`AcaoBomba` (`enum`)	4	`BOMBA_LIGOU` ou `BOMBA_DESLIGOU`

`Pilha`: histórico LIFO em **vetor estático** (64 bytes)

Campo	Tipo	Bytes	Significado
`itens`	`Evento[CAP_PILHA]`	60	Vetor de 5 eventos
`topo`	`int`	4	Índice do topo (`-1` = vazia)

`Estatisticas`: acumuladores (16 bytes)

Campo	Tipo	Bytes	Significado
`total`	`int`	4	Leituras processadas
`soma`	`float`	4	Soma das umidades (para a média)
`minimo`, `maximo`	`float`	8	Extremos observados

Variáveis globais de controle: `bomba_ligada` (estado do atuador), `proximo_id` (contador de leituras), `total_malloc` e `total_free` (auditoria de memória) e `ultima_leitura` (temporização com `millis()`).

Pior caso de memória dinâmica: 10 nós × 20 bytes = **200 bytes** de heap, limitado por `CAP_FILA`. A pilha usa 64 bytes fixos.

2. Motores da esteira FIFO (fila) e LIFO (pilha)

Por que essas estruturas:

- **Fila (FIFO) para as leituras:** o sensor produz dados em ordem temporal, e a regra de negócio precisa avaliá-los **na mesma ordem em que chegaram**. Uma fila desacopla quem produz (leitura a 1 Hz) de quem consome (processamento em lote a cada 3 leituras). Ela foi feita como **lista encadeada** para demonstrar alocação dinâmica, e o ponteiro `fim` garante a inserção em tempo constante.
- **Pilha (LIFO) para o histórico da bomba:** o evento mais importante é **o último** (o estado atual da bomba e o que o causou). Na pilha, ele fica sempre no topo, acessível em O(1). Um **vetor estático** foi escolhido porque o tamanho máximo é conhecido e não precisa de `malloc`.

Enfileiramento: `enfileirar()`, O(1)

```
int enfileirar(Fila *f, Leitura dado) {
    if (fila_cheia(f)) { /* OVERFLOW BLOQUEADO */ return 0; }  /* 1. guarda de capacidade */
    No *novo = (No *) malloc(sizeof(No));                      /* 2. aloca no heap        */
    if (novo == NULL)   { /* memória insuficiente */ return 0; }
    total_malloc++;
    novo->dado = dado;  novo->proximo = NULL;                   /* 3. preenche o nó        */
    if (fila_vazia(f)) f->inicio = novo;                        /* 4. encadeia no fim      */
    else               f->fim->proximo = novo;
    f->fim = novo;
    f->tamanho++;
    return 1;
}
```

Executa um número fixo de passos, sem laço, porque usa o ponteiro `fim`. Logo, **O(1)**.

Remoção da fila: `desenfileirar()`, O(1)

```
int desenfileirar(Fila *f, Leitura *saida) {
    if (fila_vazia(f)) { /* UNDERFLOW BLOQUEADO */ return 0; }
    No *removido = f->inicio;
    *saida    = removido->dado;             /* copia o dado antes de liberar  */
    f->inicio = removido->proximo;
    if (f->inicio == NULL) f->fim = NULL;    /* fila ficou vazia: zera o fim   */
    free(removido);                          /* devolve a memória ao heap      */
    total_free++;
    f->tamanho--;
    return 1;
}
```

Remove sempre do início, sem percorrer a lista. Logo, **O(1)**. O dado é copiado para `*saida` **antes** do `free`, então nunca se lê memória já liberada (*use-after-free*).

Empilhamento: `empilhar()`, O(1)

```
int empilhar(Pilha *p, Evento e) {
    if (pilha_cheia(p)) { /* OVERFLOW BLOQUEADO */ return 0; }  /* topo == CAP_PILHA - 1 */
    p->topo++;
    p->itens[p->topo] = e;
    return 1;
}
```

Remoção da pilha: `desempilhar()`, O(1)

```
int desempilhar(Pilha *p, Evento *saida) {
    if (pilha_vazia(p)) { /* UNDERFLOW BLOQUEADO */ return 0; }  /* topo == -1 */
    *saida = p->itens[p->topo];
    p->topo--;
    return 1;
}
```

Resumo de complexidade (Big-O)

Operação	Complexidade	Justificativa
`enfileirar`	**O(1)**	Insere direto pelo ponteiro `fim`
`desenfileirar`	**O(1)**	Remove direto pelo ponteiro `inicio`
`empilhar` / `desempilhar`	**O(1)**	Acesso direto pelo índice `topo`
`fila_vazia`, `fila_cheia`, `pilha_vazia`, `pilha_cheia`	**O(1)**	Uma comparação
`processar_fila`	**O(n)**, com n ≤ 10	Percorre cada leitura pendente uma vez
`exibir_fila` / `exibir_pilha`	**O(n)**	Percorrem a estrutura para imprimir

3. Mitigação de erros físicos (Overflow / Underflow / Memory Leak)

Risco	Onde poderia ocorrer	Como foi mitigado
Overflow da fila (exaustão do heap)	Sensor produzindo mais rápido do que o consumo	`fila_cheia()` é verificada **antes** do `malloc`. Com 10 nós, a leitura é recusada e registrada como `[OVERFLOW BLOQUEADO]`. O heap nunca passa de 200 bytes.
Falha de alocação	Heap sem espaço	O retorno do `malloc` é testado (`novo == NULL`). Em caso de falha, a função retorna 0 sem tocar em ponteiro nulo.
Underflow da fila	Remover de fila vazia	`fila_vazia()` antes de acessar `inicio`. Impede desreferenciar `NULL`.
Stack Overflow da pilha	Empilhar além do vetor	`pilha_cheia()` (`topo == CAP_PILHA - 1`) antes de escrever. Nunca escreve fora de `itens[0..4]`, então não corrompe a memória vizinha.
Stack Underflow da pilha	Desempilhar vazia	`pilha_vazia()` (`topo == -1`) antes de ler. Nunca lê `itens[-1]`.
Estouro da pilha de chamadas	Recursão ou vetores grandes em funções	O código **não tem recursão**. A pilha de eventos é um vetor global de tamanho fixo, e as variáveis locais são pequenas. O uso da pilha de chamadas é constante e previsível.
Vazamento de memória (Memory Leak)	`malloc` sem `free` correspondente	Todo nó criado em `enfileirar` é liberado em `desenfileirar`, e `processar_fila` esvazia a fila inteira a cada ciclo. Os contadores `total_malloc` e `total_free` provam isso **em tempo de execução**: o comando `e` no monitor serial mostra `malloc / free / nós na RAM`, e a diferença nunca passa de 10.
Ponteiro pendente (dangling)	Ponteiro `fim` apontando para nó liberado	Quando a fila esvazia, `fim` volta a `NULL` junto com `inicio`.
Travamento do laço principal	`delay()` bloqueante	A temporização usa `millis()`. O `loop()` nunca para, e os comandos seriais são atendidos a qualquer momento.

Todas as funções de inserção e remoção **retornam 0 ou 1** para quem as chamou, que só avança o contador (`proximo_id++`) quando a operação deu certo.

Prova ao vivo pelo monitor serial: `f` mostra a fila, `p` mostra a pilha, `d` desempilha (inclusive forçando o underflow) e `e` mostra as estatísticas e o balanço `malloc`/`free`.

V. Topologia de Conexão (pinagem)

Componente	Pino do componente	Pino da ESP32	Tipo
Sensor capacitivo v1.2	VCC	3V3	Alimentação
Sensor capacitivo v1.2	GND	GND	Terra
Sensor capacitivo v1.2	AOUT (sinal)	**GPIO35**	Entrada analógica (ADC1, 12 bits, atenuação 11 dB, faixa 0 a ~3,3 V)
Módulo relé	VCC	5V (VIN)	Alimentação
Módulo relé	GND	GND	Terra
Módulo relé	IN	**GPIO26**	Saída digital (HIGH = bomba ligada)
Mini bomba	+ / −	contato NA e COM do relé, em série com a fonte	Carga chaveada
LED de status	—	**GPIO2** (LED embutido)	Saída digital (aceso = bomba ligada)
Computador	USB	USB da placa (CP210x, COM4)	Alimentação + serial a 115200 baud

Observações de engenharia:

- O GPIO35 pertence ao **ADC1**, que continua funcionando com o Wi-Fi ligado (o ADC2 não). Isso deixa caminho livre para uma versão com envio por Wi-Fi.
- O GPIO35 é **somente entrada**, adequado para sensor.
- O sensor foi movido do GPIO34 para o GPIO35 porque o GPIO34 da placa usada apresentou defeito (leitura fixa em ~690 sem nada conectado). Na simulação do Wokwi, o potenciômetro permanece no GPIO34.

VI. Manual de Deploy

Dependências (bibliotecas incluídas)

Dependência	Versão	Uso
`<stdlib.h>` (biblioteca padrão C)	—	`malloc`, `free`, `NULL`
Núcleo Arduino para ESP32 (`esp32:esp32`, Espressif)	3.3.4	`Serial`, `analogRead`, `analogReadResolution`, `analogSetPinAttenuation`, `digitalWrite`, `pinMode`, `millis`

Não há bibliotecas de terceiros. A planta de execução em C puro (`codigo-c/aquasense.c`) usa apenas `<stdio.h>`, `<stdlib.h>` e `<time.h>`.

Parâmetros de comunicação serial

Parâmetro	Valor
Baud Rate	**115200**
Formato	8 bits de dados, sem paridade, 1 bit de parada (8N1)
Fim de linha	`\n`

Passo a passo de gravação

1. Instalar o **Arduino IDE 2.x** e, no Gerenciador de Placas, o pacote **esp32 by Espressif Systems** (3.3.4).
2. Abrir `aquasense_esp32/aquasense_esp32.ino`.
3. Selecionar a placa **ESP32 Dev Module** e a porta COM da placa.
4. Clicar em **Carregar**. Se aparecer `Connecting...` e a gravação não começar, segurar **BOOT**, apertar e soltar **EN**, e soltar **BOOT**.
5. Ao terminar, apertar **EN** para reiniciar a placa e executar o programa.
6. Abrir o **Monitor Serial a 115200 baud**. Comandos: `f` (fila), `p` (pilha), `d` (desempilhar), `e` (estatísticas e memória).

Pela linha de comando, como alternativa:

```
arduino-cli compile --fqbn esp32:esp32:esp32 aquasense_esp32
arduino-cli upload  --fqbn esp32:esp32:esp32 -p COM4 aquasense_esp32
```

Painel de monitoramento (opcional)

Abrir `medidor.html` no **Google Chrome**, clicar em conectar e escolher a porta da ESP32. A página usa a Web Serial API, sem servidor nem internet. A ESP32 envia 10 linhas por segundo no formato:

```
DADO;id=12;adc=2870;umid=42.5;bomba=0
```

Com o painel aberto, o Chrome ocupa a porta serial. Para gravar de novo, é preciso fechar o painel antes.

Planta de execução em C (sem hardware)

```
gcc -Wall -Wextra -o aquasense codigo-c/aquasense.c
./aquasense
```

Simulação (sem hardware)

Wokwi: instruções em `wokwi/LEIAME.md`. Um potenciômetro substitui o sensor.

Constantes configuráveis (`#define`)

Constante	Valor	Significado
`LIMITE_SECO`	30.0	Liga a bomba abaixo deste percentual
`LIMITE_ENCHARCADO`	60.0	Desliga a bomba acima deste percentual
`CAP_FILA`	10	Capacidade máxima da fila
`CAP_PILHA`	5	Capacidade máxima da pilha
`ADC_SECO` / `ADC_IDEAL` / `ADC_MOLHADO`	4095 / 2411 / 1277	Pontos de calibração
`INTERVALO_LEITURA`	1000 ms	Leitura que entra na fila (1 Hz)
`INTERVALO_ENVIO`	100 ms	Envio para o painel (10 Hz)
`PROCESSAR_A_CADA`	3	Processa a fila a cada 3 leituras
`AMOSTRAS_POR_LEITURA`	16	Média de amostras do ADC (filtro de ruído)

Estrutura do repositório

Caminho	Conteúdo
`aquasense_esp32/aquasense_esp32.ino`	Firmware da placa real (v1.1)
`medidor.html`	Painel de monitoramento via Web Serial
`codigo-c/aquasense.c`	Planta de execução em C puro (v1.0)
`wokwi/`	Simulação no Wokwi (`sketch.ino` , `diagram.json` , `LEIAME.md`)
`index.html`	Site de demonstração (GitHub Pages)
`docs/AquaSense - Documentacao Tecnica - Grupo 7.pdf`	Esta documentação em PDF

AquaSense v1.1, Grupo 7, Inovatech 2026.